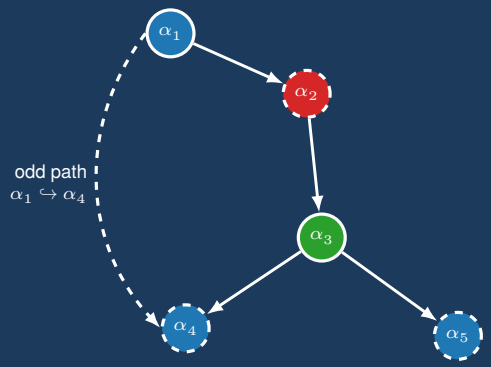
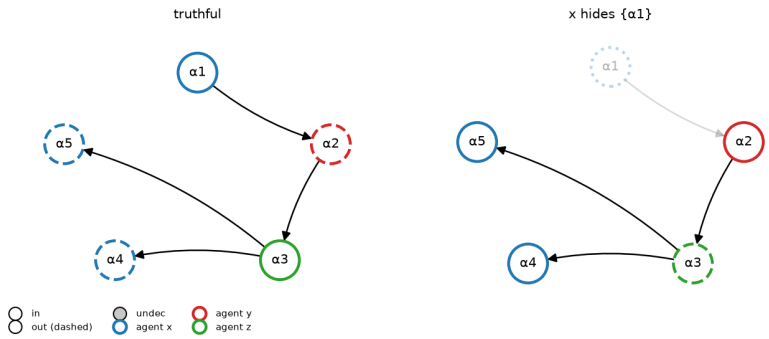
Autonomous Systems • Assignment 3 Report

Argumentation Engine and Strategic Argumentation

Dung semantics, labellings and acceptance; the grounded direct argumentation mechanism, and when agents gain by hiding arguments



Abstract. We implement `argengine`, an engine for Dung-style abstract argumentation following Chapter 5 of Weiss, *Multiagent Systems*: conflict-freeness, defence, the characteristic function, admissibility and completeness, the grounded, complete, preferred, stable and semi-stable extensions, all legal labellings and the acceptance status of an argument. On top of it we implement the **grounded direct argumentation mechanism** with best responses, an exact strategyproofness (Theorem 5.3) check and the Theorem 5.4 condition, correct on cyclic graphs. All examples of the chapter are reproduced, including the two misprints in the brief. On 3,000 random 3-agent instances the share of strategyproof mechanisms falls from **0.628** ($p = 0.15$) to **0.105** ($p = 0.35$). **No** instance satisfies the Theorem 5.4 condition and is manipulable; we explain why. The 876 strategyproof instances that violate the condition are all explained by three mechanisms: *offset*, *self-neutralised* and *dormant*.



Example 5.10 drawn by the engine. Left: all agents reveal everything, so x gets only α_1 accepted. Right: x hides α_1, and α_4, α_5 become accepted, so lying pays.

Submitted by (group)

Nisarg Upadhyay	B23CS1075
Mohit Meemrauth	B23CS1038
Divyansh Maheshwari	B23CM1015
Tejas Kalkar	B23CM1051

November 2026

Contents

1 Introduction **1**

2 Background **1**

3 Part 1: the argumentation engine **2**
3.1 Framework class and basic tests (items 1–5) 2
3.2 Semantics, labellings and acceptance (items 6A–6E) 2

4 Validation on the book's examples **3**

5 Part 2: the grounded direct argumentation mechanism **4**
5.1 Mechanism, best responses and strategyproofness (items 1–3) 4
5.2 The Theorem 5.4 condition on cyclic graphs (item 4) 4
5.3 Examples 5.10 and 5.11 4

6 Experiments **5**
6.1 Q1: fraction of strategyproof instances 5
6.2 Q2: condition holds but not strategyproof – none 6
6.3 Q3: strategyproof although the condition fails 6

7 Demonstration **7**

8 Conclusion **7**

References **7**

1 Introduction

An *argumentation framework* keeps only which argument attacks which; a *semantics* decides which sets of arguments can be accepted together (Dung 1995). Chapter 5 of Weiss's *Multiagent Systems* (Rahwan) develops this theory and then asks a mechanism-design question: if self-interested agents each hold some of the arguments and the outcome is the grounded extension, can an agent gain by *hiding* arguments?

Part 1 of the assignment builds an argumentation engine; Part 2 implements the grounded direct argumentation mechanism and studies, on random instances, how often it is strategyproof and how well Theorem 5.4 predicts this.

Headline results

- **All book examples are reproduced**, and both misprints named in the brief are confirmed: in Example 5.10, u_y and u_z are printed with each other's argument sets; after Example 5.11, the argument that cancels x's indirect self-defeat is α_6, not α_5.
- **Q1.** Strategyproof fraction: **0.628**, **0.284** and **0.105** for $p = 0.15, 0.25, 0.35$.
- **Q2. None.** No instance satisfies the Theorem 5.4 condition yet fails strategyproofness, under any reading of "odd-length path". A further search over 5,625,390 instances agrees, and we prove why.
- **Q3. Yes, 876 instances** (82–97 % of all strategyproof ones), all explained: 856 *offset*, 2 *self-neutralised* (as in Example 5.11), 18 *dormant*.

Table 1. Assignment items and where they are met.

Requirement	Function	Section	
1 Framework class $\langle \mathcal{A}, \rightharpoonup \rangle$	`ArgumentationFramework` (alias `AF`)	§3.1	✓
2 Conflict-free test	`is_conflict_free(af, S)`	§3.1	✓
3 Is α defended by S?	`defends(af, S, a)`	§3.1	✓
4 Characteristic function	`characteristic_function(af, S)`	§3.1	✓
5 Admissible / complete	`is_admissible`, `is_complete_extension`	§3.1	✓
6A Grounded extension	`grounded_extension`, `grounded_trace`	§3.2	✓
6B All complete extensions	`complete_extensions`	§3.2	✓
6C Preferred, stable, semi-stable	`preferred_/stable_/semi_stable_extensions`, `all_extensions`	§3.2	✓
6D All legal labellings	`all_labellings`, `is_legal`	§3.2	✓
6E Acceptance status	`acceptance_status(af, a, semantics)`	§3.2	✓
Test on the book's examples	161 automated tests	§4	✓
S1 Mechanism	`grounded_mechanism(R, reports)`	§5	✓
S2 Best responses	`best_responses(R, types, i, others)`	§5	✓
S3 Theorem 5.3 check	`is_strategyproof(R, types)`	§5	✓
S4 Theorem 5.4 condition (with cycles)	`theorem_5_4_condition(R, types, mode)`	§5.2	✓
Experiments Q1, Q2, Q3	`experiments/run_experiments.py`, `analyse.py`	§6	✓
Book misprints	confirmed by the engine and by tests	§5.3	✓
Code, Readme, Report, Demonstration	`argengine/`, `README.md`, this report, `demo/`	§7	✓

2 Background

Definitions follow Weiss (ed.), *Multiagent Systems*, 2nd ed., Ch. 5. In the code, α_k is written `"ak"`.

Abstract argumentation (Defs. 5.1–5.8)

An argumentation framework is $\langle \mathcal{A}, \rightharpoonup \rangle$ with a finite set of arguments $\mathcal{A}$ and an attack relation $\rightharpoonup \subseteq \mathcal{A} \times \mathcal{A}$. Let α^- be the attackers of α and S^+ the arguments attacked by S.

- S is **conflict-free** iff $S \cap S^+ = \emptyset$; S **defends** α iff $\alpha^- \subseteq S^+$; the **characteristic function** is $F(S) = \{\alpha \mid S \text{ defends } \alpha\}$.
- A conflict-free S is **admissible** iff $S \subseteq F(S)$ and **complete** iff $S = F(S)$.
- **Grounded**: the least complete extension (least fixed point of F). **Preferred**: maximal complete extensions.

Stable: $S^+ = \mathcal{A} \setminus S$. **Semi-stable**: complete extensions with maximal range $S \cup S^+$.

- A **legal labelling** $L : \mathcal{A} \to \{\mathsf{in}, \mathsf{out}, \mathsf{undec}\}$: α is out iff some attacker is in, in iff all attackers are out, otherwise undec. Legal labellings correspond one to one to complete extensions (in $= S$, out $= S^+$).
- α is **skeptically accepted** if it is in every extension, **credulously accepted** if in some, and **rejected** if in none.

Strategic argumentation (Defs. 5.17–5.22, Theorems 5.3, 5.4)

Agent i holds a *type* $\mathcal{A}_i \subseteq \mathcal{A}$ and reports any subset $\mathcal{A}_i^\circ \subseteq \mathcal{A}_i$ (agents lie only by hiding). The outcome is the grounded extension of the graph induced by $\bigcup_i \mathcal{A}_i^\circ$, and the utility is $u_i(o) = |\mathcal{A}_i \cap o|$. The mechanism is **strategyproof** iff revealing $\mathcal{A}_i$ is a best response for every agent against every combination of the others' reports (the condition checked for Theorem 5.3). **Theorem 5.4:** if no agent holds α, β with an *odd-length* directed path $\alpha \rightsquigarrow \beta$ (an indirect defeat $\alpha \hookrightarrow \beta$; length 1 is a direct attack), the mechanism is strategyproof.

3 Part 1: the argumentation engine

3.1 Framework class and basic tests (items 1–5)

`ArgumentationFramework(arguments, attacks)` stores $\langle \mathcal{A}, \rightharpoonup \rangle$. The API takes and returns ordinary Python sets of names. Internally, argument α_i is bit i and every set is an integer *bitmask*; for each argument the class precomputes the masks of its attackers and targets. The basic tests then become a few bit operations:

- `is_conflict_free(af, S)`: $S^+ \wedge S = 0$, where S^+ is the OR of the target masks of S;
- `defends(af, S, a)`: `attackers`$[\alpha] \wedge \neg S^+ = 0$;
- `characteristic_function(af, S)`: all α with that property, i.e. $F(S)$;
- `is_admissible`: conflict-free and $S \subseteq F(S)$; `is_complete_extension`: conflict-free and $S = F(S)$.

Because every subset of $\mathcal{A}$ is an integer in $[0, 2^n)$, subsets can be enumerated and results cached cheaply, which Part 2 relies on. Unknown arguments raise an error, and `restrict(S)` returns the induced sub-framework.

3.2 Semantics, labellings and acceptance (items 6A–6E)

Algorithms 1–2 – grounded and complete extensions

1 **Grounded:** $S \leftarrow \emptyset$; **repeat** $S \leftarrow F(S)$ **until** S does not change — *at most $n+1$ rounds*
2 **Complete:** $G \leftarrow$ grounded; *free* $\leftarrow \mathcal{A} \setminus (G \cup G^+)$
3 **for** every $X \subseteq$ *free*: **if** $S = G \cup X$ is conflict-free **and** $F(S) = S$ **then** output S

- **6A Grounded.** F is monotone, so iterating it from $\emptyset$ reaches the least fixed point. `grounded_trace` returns $F^1(\emptyset), F^2(\emptyset), \ldots$ as in the book.
- **6B Complete.** Every complete extension contains the grounded extension G and, being conflict-free, nothing in G^+; so only subsets of the remaining "free" arguments must be tried. This is exact and much smaller than 2^n.
- **6C Preferred / stable / semi-stable** are selected from the complete extensions: the $\subseteq$-maximal ones; those with $S^+ = \mathcal{A} \setminus S$; and those with $\subseteq$-maximal range $S \cup S^+$. `all_extensions` returns all five semantics at once.
- **6D Labellings.** Each complete extension S gives the labelling in $= S$, out $= S^+$, undec $=$ the rest. `is_legal` checks the definition directly, and a 3^n brute force confirms that no legal labelling is missed. Table 5.1 is implemented as selectors (min/max in, min/empty undec).
- **6E Acceptance status.** `acceptance_status(af, a, semantics)` returns skeptically accepted, credulously accepted or rejected for any of the five semantics; `status_table` lists all arguments.

Design decision: no extension

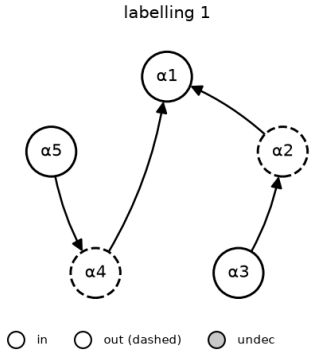
Under stable semantics an odd cycle has *no* extension, so "in every extension" is vacuously true. We report such arguments as **rejected** (with `n_extensions = 0`), not as skeptically accepted.

4 Validation on the book's examples

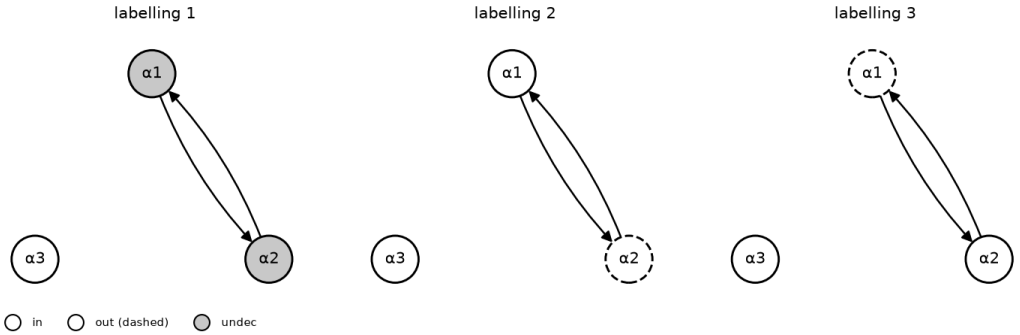
Each row of Table 2 is an automated test (161 tests in total, `pytest`) and is printed by the demonstration.

Table 2. Book statements and engine output (identical).

Example	Book statement = engine output	
Fig. 5.1, Ex. 5.5	$\{\alpha_1, \alpha_3, \alpha_5\}$ conflict-free, $\{\alpha_1, \alpha_2\}$ not; $\emptyset, \{\alpha_3\}, \{\alpha_5\}, \{\alpha_3, \alpha_5\}, \{\alpha_1, \alpha_3, \alpha_5\}$ admissible	✓
Fig. 5.1, Ex. 5.6	$\{\alpha_3, \alpha_5\}$ defends α_1, $\{\alpha_3\}$ does not; $F(\{\alpha_3\}) = F(\{\alpha_5\}) = \{\alpha_3, \alpha_5\}$; $\{\alpha_3, \alpha_5\}$ admissible but not complete; only complete extension $\{\alpha_1, \alpha_3, \alpha_5\}$	✓
Fig. 5.1, p. 184	$F^1(\emptyset) = \{\alpha_3, \alpha_5\}$, $F^2(\emptyset) = F^3(\emptyset) = \{\alpha_1, \alpha_3, \alpha_5\}$ = grounded	✓
Fig. 5.2, Ex. 5.7, 5.8	complete: $\{\alpha_3\}, \{\alpha_1, \alpha_3\}, \{\alpha_2, \alpha_3\}$ (three labellings); grounded $\{\alpha_3\}$; preferred $\{\alpha_1, \alpha_3\}, \{\alpha_2, \alpha_3\}$; α_3 skeptically, α_1, α_2 credulously accepted	✓
Fig. 5.3, Ex. 5.9	grounded $= \{a, c, e\}$ ($F^1 = \{e\}$, $F^2 = \{c, e\}$, $F^3 = \{a, c, e\}$)	✓
Ex. 5.10, 5.11	see Table 5	✓



(a) Fig. 5.1: grounded $\{\alpha_1, \alpha_3, \alpha_5\}$.



(b) Fig. 5.2: the three legal labellings.

Figure 1. Book frameworks drawn by the engine: solid border = in, dashed = out, grey = undec.

The book's frameworks do not separate preferred, stable and semi-stable, so we added the frameworks of Table 3; Table 4 shows that the acceptance status changes with the semantics as expected. Property-based tests (*hypothesis*, hundreds of random frameworks) also check that grounded $\subseteq$ every complete extension, stable $\subseteq$ semi-stable $\subseteq$ preferred $\subseteq$ complete, preferred = maximal admissible, and that the labellings agree with the 3^n brute force.

Table 3. Extra frameworks ("none" = no extension exists).

Framework	Attacks	Complete	Preferred	Stable / semi-stable
Odd 3-cycle	$a \rightharpoonup b \rightharpoonup c \rightharpoonup a$	$\emptyset$	$\emptyset$	none / $\emptyset$
Preferred $\neq$ semi-stable	$a \rightleftarrows b$, $b \rightharpoonup c$, $c \rightharpoonup d \rightharpoonup e \rightharpoonup c$	$\emptyset, \{a\}, \{b, d\}$	$\{a\}, \{b, d\}$	$\{b, d\}$ / $\{b, d\}$
Semi-stable without stable	$a \rightleftarrows b$, $b \rightharpoonup c$, $c \rightharpoonup c$, $d \rightharpoonup e \rightharpoonup f \rightharpoonup d$	$\emptyset, \{a\}, \{b\}$	$\{a\}, \{b\}$	none / $\{b\}$

Table 4. Item 6E: acceptance status by semantics (S = skeptically, C = credulously accepted, R = rejected).

Framework	Argument	grounded	complete	preferred	stable	semi-stable
Fig. 5.2	α_1, α_2	R	C	C	C	C
	α_3	S	S	S	S	S
Pref. $\neq$ semi-stable	a	R	C	C	R	R
	b, d	R	C	C	S	S
Semi-stable w/o stable	b	R	C	C	R (none)	S

5 Part 2: the grounded direct argumentation mechanism

5.1 Mechanism, best responses and strategyproofness (items 1–3)

- **Mechanism.** `grounded_mechanism(R, reports)` builds the graph induced by the union of the revealed sets under the global attack relation `R` and returns its grounded extension (Part 1 code). `MechanismInstance` caches the outcome of every revealed set (at most $2^n = 256$ for $n = 8$), so repeated queries are look-ups.
- **Best responses.** `best_responses(R, types, i, others)` enumerates all $2^{|\mathcal{A}_i|}$ reports of agent i against the fixed reports of the others and returns *every* utility-maximising report, plus whether truth-telling is among them (ties are common).
- **Strategyproofness.** `is_strategyproof(R, types)` runs Algorithm 3 and returns a *witness* (the others' reports, what the agent hides, and both utilities) for every manipulable agent.

Algorithm 3 – strategyproofness check (Theorem 5.3 condition)

for each agent i, **for** every $O \subseteq \bigcup_{j \neq i} \mathcal{A}_j$ — *every possible union of the others' reports*
  $u_T \leftarrow |\mathcal{A}_i \cap g(\mathcal{A}_i \cup O)|$ — *truthful utility*
  if some $S \subseteq \mathcal{A}_i$ has $|\mathcal{A}_i \cap g(S \cup O)| > u_T$ **then** record a witness
return strategyproof $\iff$ no witness

The outcome depends on the reports only through their union, and the others can produce exactly every subset of the union of their types, so this covers every combination of the others' reports. A brute-force version that enumerates the profiles agent by agent agrees with it in the tests.

5.2 The Theorem 5.4 condition on cyclic graphs (item 4)

`theorem_5_4_condition` checks the *full* graph for two arguments of the same agent joined by an odd-length path, and returns each violating path. On a cyclic graph, "odd-length path" has two readings, and a naive path search may loop forever; both readings are implemented safely:

- **Odd simple path** (no repeated vertex; default, the literal graph-theory meaning): DFS with a visited set, so it always terminates; exact for $n \leq 8$.
- **Odd walk** (vertices may repeat): BFS over states (vertex, parity of the length). Each state is visited once, so cycles cannot cause non-termination; an odd cycle just makes a vertex reachable with both parities.

A flag `include_self` decides whether $\alpha = \beta$ (a self-attack or an odd cycle through one owned argument) counts. The experiments report all four combinations.

5.3 Examples 5.10 and 5.11

Table 5. Examples 5.10 and 5.11. Types $x = \{\alpha_1, \alpha_4, \alpha_5\}$ (+α_6 in 5.11), $y = \{\alpha_2\}$, $z = \{\alpha_3\}$; attacks $\alpha_1 \rightharpoonup \alpha_2 \rightharpoonup \alpha_3$, $\alpha_3 \rightharpoonup \alpha_4$, $\alpha_3 \rightharpoonup \alpha_5$ (+$\alpha_6 \rightharpoonup \alpha_3$ in 5.11).

	Example 5.10	Example 5.11
Truthful outcome	$\{\alpha_1, \alpha_3\}$	$\{\alpha_1, \alpha_4, \alpha_5, \alpha_6\}$
Utilities u_x, u_y, u_z	1, 0, 1	4, 0, 0
x's best response	$\{\alpha_4, \alpha_5\}$ (**hide** α_1), utility 2	reveal everything, utility 4
Strategyproof?	✗ (x hides α_1: outcome $\{\alpha_2, \alpha_4, \alpha_5\}$)	✓
Thm 5.4 condition	fails: $\alpha_1 \rightharpoonup \alpha_2 \rightharpoonup \alpha_3 \rightharpoonup \alpha_4$ (and α_5)	fails: same paths

In Example 5.10, α_1 defeats α_2, which reinstates α_3, which defeats x's own α_4 and α_5; hiding α_1 loses one accepted argument and gains two (title-page figure). In Example 5.11, α_6 defeats α_3 whatever happens to α_1, so hiding no longer helps: the mechanism is strategyproof even though the condition fails, the first example of Q3.

Misprints in the book (confirmed)

- **Example 5.10:** the book prints $u_y(o, \{\alpha_3\}) = 1$ and $u_z(o, \{\alpha_2\}) = 0$; the correct values are $u_y(o, \{\alpha_2\}) = 0$ and $u_z(o, \{\alpha_3\}) = 1$ for $o = \{\alpha_1, \alpha_3\}$.
- **After Example 5.11:** the argument that cancels x's indirect self-defeat is α_6, not α_5. With α_6 in x's type the mechanism is strategyproof; without it, it is not. α_5 attacks nothing, so it cannot cancel anything.

6 Experiments

Setup. 3 agents; $n \sim U\{6, 7, 8\}$ arguments; each ordered pair (α, β), $\alpha \neq \beta$, is an attack with probability $p \in \{0.15, 0.25, 0.35\}$; each argument is given to one agent uniformly at random (types are disjoint and may be empty); **1000 instances per** p, seed 2026. For each instance we record whether the mechanism is strategyproof (Algorithm 3) and whether the Theorem 5.4 condition holds under each reading. Results are exact; the whole run takes about 3 s. Raw data: `experiments/results/instances.csv`.

6.1 Q1: fraction of strategyproof instances

Table 6. Q1 – strategyproofness by attack probability (95 % Wilson intervals).

p	SP	fraction	95 % CI	$n = 6 / 7 / 8$	acyclic	cyclic	Thm 5.4 holds
0.15	628	**0.628**	[0.598, 0.657]	0.74 / 0.62 / 0.52	0.89	0.34	0.111
0.25	284	**0.284**	[0.257, 0.313]	0.43 / 0.28 / 0.14	0.87	0.18	0.027
0.35	105	**0.105**	[0.087, 0.126]	0.22 / 0.08 / 0.03	0.89	0.08	0.003

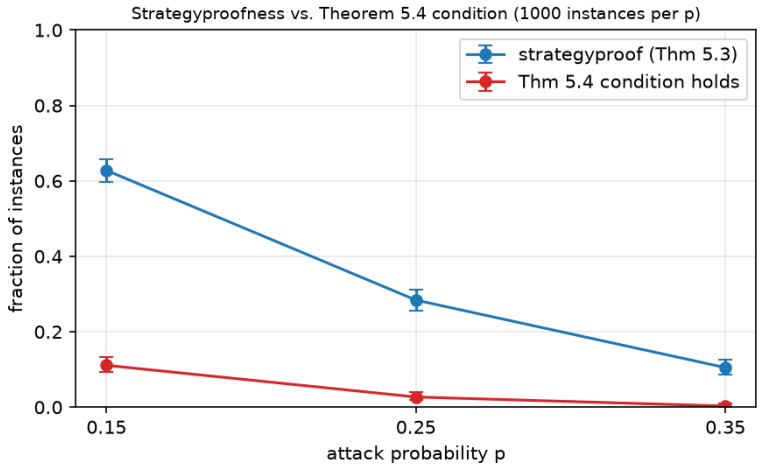


Figure 2. Q1: fraction of strategyproof instances and of instances satisfying the Theorem 5.4 condition.

Q1 – findings

- Strategyproofness becomes rare as the graph gets denser (63 % → 28 % → 11 %) and as n grows: more attacks create more odd paths between arguments of the same agent, and such a path is needed to gain by hiding.
- Cycles drive the effect: acyclic instances are strategyproof 87–89 % of the time at every p, cyclic ones only 8–34 %; the share of cyclic graphs grows from 47 % to 97 %.
- The Theorem 5.4 condition holds far less often than strategyproofness (11.1 %, 2.7 %, 0.3 %).

6.2 Q2: condition holds but not strategyproof – none

Table 7. Theorem 5.4 condition × strategyproofness, for $p = 0.15 / 0.25 / 0.35$. Q2 = condition holds ∧ not SP.

Reading of "odd-length path"	cond ∧ SP	Q2	Q3: ¬cond ∧ SP	¬cond ∧¬SP
simple path, $\alpha = \beta$ allowed (**primary**)	111 / 27 / 3	**0 / 0 / 0**	517 / 257 / 102	372 / 716 / 895
walk, $\alpha = \beta$ allowed	111 / 27 / 3	**0 / 0 / 0**	517 / 257 / 102	372 / 716 / 895
simple path, $\alpha \neq \beta$	113 / 29 / 3	**0 / 0 / 0**	515 / 255 / 102	372 / 716 / 895
walk, $\alpha \neq \beta$	112 / 27 / 3	**0 / 0 / 0**	516 / 257 / 102	372 / 716 / 895

No instance satisfies the condition yet fails strategyproofness, under any reading. To rule out luck, `experiments/stress_test.py` also checked *every* 3-agent instance with $n \leq 4$ arguments (5,322,390 instances, self-attacks allowed) and 300,000 further random ones, using the weakest reading: again **0** counterexamples. This is expected, because Theorem 5.4 holds on cyclic graphs, even in a sharper form:

Lemma and sharpened Theorem 5.4

Lemma. If hiding S from the revealed set M turns an argument β in ($\beta \in GE(M \setminus S) \setminus GE(M)$), then some $\alpha \in S$ has an odd *simple* path $\alpha \rightsquigarrow \beta$ in the graph induced by M.

Theorem. If no agent holds two *distinct* arguments joined by an odd simple path in the full graph, the mechanism is strategyproof.

Proof sketch. $\beta \notin GE(M)$, so some attacker γ of β is not defeated by $GE(M)$. If $\gamma \in S$, the path $\gamma \rightharpoonup \beta$ has length 1. Otherwise γ is defeated in $GE(M \setminus S)$ by some δ that is itself newly in, at an earlier stage of the fixed-point iteration; by induction an odd path leads from S to δ, and $\delta \rightharpoonup \gamma \rightharpoonup \beta$ adds two edges, keeping it odd. Repeated vertices can only be γ's, and removing the loop between them removes an even number of edges, so the path can be made simple. For the theorem: a gaining agent has some own β newly in, so the lemma gives an odd path from one of its own hidden $\alpha \neq \beta$ to β, a contradiction. ■

All four readings in Table 7 forbid at least as much as this one, so **Q2 must be empty**. The lemma was also confirmed on the witness of all 1983 manipulable instances.

6.3 Q3: strategyproof although the condition fails

Table 8. Q3 – strategyproof instances that violate the (primary) Theorem 5.4 condition, by explanation.

p	SP but cond. fails	share of all SP	offset	self-neutralised	dormant
0.15	517	0.823	506	2	9
0.25	257	0.905	249	0	8
0.35	102	0.971	101	0	1
total	**876**		**856**	**2**	**18**

Yes, and they are the norm. Theorem 5.4 only asks whether an odd path *exists*; it ignores whether hiding actually makes the target in, and whether the gain outweighs the loss. `classify_agent` checks every revealed set and every hidden set and puts each violating agent in one of three classes:

- **Offset (856).** Hiding does free one of the agent's arguments, but the agent always loses at least as many, usually the hidden argument itself, so the net gain is ≤ 0. In 472 of these instances the only violation is a direct attack inside a type: in #224, y owns $\alpha_1 \rightharpoonup \alpha_4$ and can trade α_1 for α_4 but never gain. In #254, $x = \{\alpha_2, \alpha_4\}$ with $\alpha_2 \rightharpoonup \alpha_6 \rightharpoonup \alpha_1 \rightharpoonup \alpha_4$: hiding α_2 frees α_4 but loses α_2. Example 5.10 is the case where the trade is *not* balanced (hiding one argument frees two).
- **Self-neutralised (2).** The Example 5.11 pattern: hiding helps only if the agent also withholds another of its own arguments, which otherwise blocks the gain. In #296, $z = \{\alpha_1, \alpha_2, \alpha_4\}$ has $\alpha_4 \rightharpoonup \alpha_5 \rightharpoonup \alpha_6 \rightharpoonup \alpha_1$; hiding α_4 would free α_1 only while z withholds α_2, since $\alpha_2 \rightharpoonup \alpha_6$ keeps α_6 out anyway (Figure 3).
- **Dormant (18).** The odd path never changes acceptance: (i) an odd cycle through a single owned argument ($\alpha = \beta$; hiding it removes the argument itself, #203); (ii) the path runs into an odd cycle that keeps the target undec (#970); (iii) hiding only turns the target from out into undec, which gives no utility (#900).

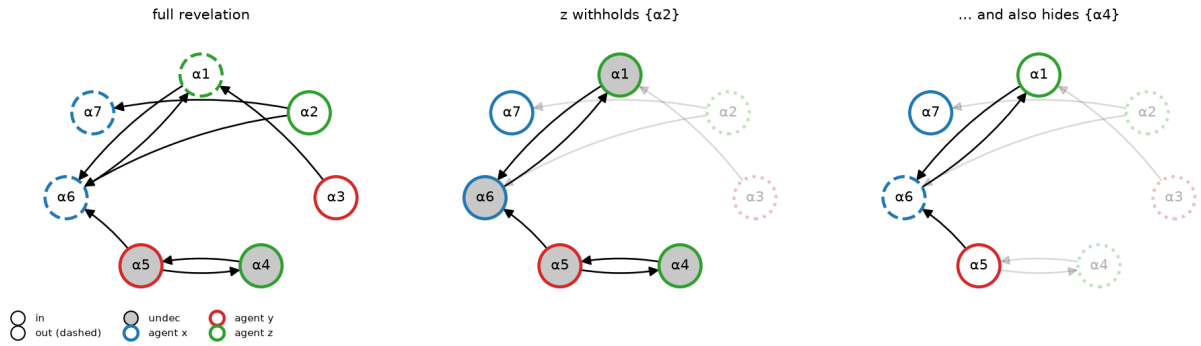


Figure 3. Instance #296 (self-neutralised). Left: full revelation. Middle: z withholds α_2. Right: only then does hiding α_4 turn α_1 in. With α_2 revealed, α_6 is always out, so hiding α_4 is useless. Border colour = owner.

Q2 / Q3 – findings

- Theorem 5.4 is **sound** (no Q2 instance, and a proof) but **very conservative**: 82–97 % of the strategyproof instances violate it.
- Nearly all of these (856 of 876) are *offset*: a gain exists but costs as much as it gives. The exact Theorem 5.3 check, not the graph condition, characterises strategyproofness.

7 Demonstration

```
python -m pytest -q                                                  161 tests
python -m argengine.demo --part 1                                    items 1–6E on all book examples
python -m argengine.demo --part 2                                    Examples 5.10, 5.11 and the misprints
python -m argengine.demo --file demo/inputs/custom_strategic.json    any framework (JSON / .apx)
python experiments/run_experiments.py && python experiments/analyse.py   Q1–Q3
```

The CLI walks through every item: the basic tests on Figure 5.1, the grounded trace, all extensions, labellings and an acceptance-status table for every framework; then utilities, best responses, the strategyproofness verdict with a witness and the Theorem 5.4 violations for Examples 5.10 and 5.11. `demo/demo.ipynb` (executed, with drawings) and `demo/demo.html` show the same in notebook form. Setup and all commands are in `README.md`.

8 Conclusion

`argengine` implements every function asked for in Part 1 and reproduces all examples of Chapter 5. The grounded direct mechanism has exact best responses, an exact strategyproofness check with witnesses, and the Theorem 5.4 condition under both readings of "odd-length path", safe on cycles. Strategyproofness is common for sparse attack graphs (63 % at $p = 0.15$) and rare for dense ones (11 % at $p = 0.35$). Theorem 5.4 is never wrong but is very conservative; every strategyproof instance that violates it is explained by an offset, self-neutralising or dormant mechanism. **Limitations:** enumeration is exponential in n (fine for $n \leq 8$; larger frameworks would need SAT/ASP solvers), and agents can only hide arguments, as in the book.

References

[1] G. Weiss (ed.), *Multiagent Systems*, 2nd ed., MIT Press, 2013 – Ch. 5, I. Rahwan, "Argumentation among Agents".

[2] P. M. Dung, "On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games", *Artificial Intelligence* 77(2), 1995.

[3] M. Caminada, "Semi-stable semantics", *Proc. COMMA*, 2006.

[4] I. Rahwan, K. Larson, F. Tohmé, "A characterisation of strategy-proofness for grounded argumentation semantics", *Proc. IJCAI*, 2009.